Technical Leadership Community Brazil

IBM Open Innovation Community Affiliate

Mini Paper Series Ano 21
Outubro, 2026 – Nº 423

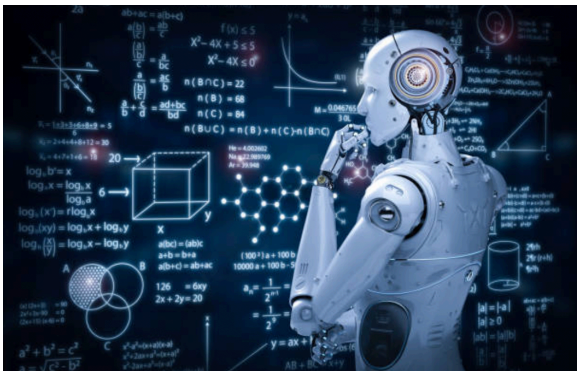
GPT do Zero: Implementando um Modelo de Linguagem *Transformer*

Mateus de Barros

Todo mundo já conversou com um assistente de IA e se perguntou: como uma máquina consegue "entender" o que escrevemos e responder de forma coerente? Por trás dessa mágica aparente existe um mecanismo chamado *Transformer*, a arquitetura que deu origem ao GPT (*Generative Pre-trained Transformer*) e a praticamente toda a geração atual de modelos de linguagem. Este texto conta a experiência de construir uma versão mínima desse modelo do absoluto zero, usando apenas operações matemáticas básicas, sem nenhuma biblioteca pronta de aprendizado profundo.

O objetivo não era competir com nada que já existe no mercado, mas abrir a caixa-preta e entender, peça por peça, o que realmente acontece lá dentro.

O primeiro obstáculo é que computadores não leem letras, leem números. A solução mais simples é dar um número para cada caractere possível do texto, um pouco como organizar um alfabeto numa lista telefônica, onde cada letra tem seu próprio índice. É um método rústico se comparado ao que sistemas comerciais usam, mas tem uma vantagem enorme para fins didáticos: nada fica escondido atrás de vocabulários pré-processados.



O coração do *Transformer* é um mecanismo chamado *self-attention*, ou atenção. A melhor forma de entender isso é imaginar uma roda de conversa em uma festa: cada pessoa (cada palavra da frase) precisa decidir, a cada momento, para quem prestar atenção entre todas as outras pessoas presentes. Uma palavra como "banco" presta muita atenção em "rio" se a frase for sobre pesca, e presta atenção em "dinheiro" se a frase for sobre finanças. O modelo aprende, sozinho, a calcular esse grau de relevância entre cada palavra e todas as outras, e é exatamente esse cálculo, feito à mão em matrizes, que revela por que a atenção é tão custosa computacionalmente: cada palavra precisa "perguntar" a todas as outras, o tempo todo.

Duas peças completam esse quebra-cabeça. A primeira é a codificação posicional: como o mecanismo de atenção olha para todas as palavras ao mesmo tempo, sem noção de ordem, é como entregar para o modelo as páginas de um livro fora de ordem. Por isso, cada posição na frase recebe também uma marcação numérica própria, como se fossem números de página costurados ao texto. A segunda peça é a máscara causal: como o modelo aprende prevendo a próxima palavra, ele não pode "espiar" o que ainda vai ser dito, da mesma forma que alguém lendo um romance não deveria pular para o último capítulo antes de terminar o primeiro.

Para testar o modelo na prática, ele foi treinado com trechos de domínio público de três autores brasileiros e portugueses: Machado de Assis, Fernando Pessoa e Guimarães Rosa. Trata-se de um corpus pequeno, escolhido de propósito, com pouco mais de mil e novecentos caracteres originais, suficiente para observar o comportamento do modelo sem exigir dias de processamento.

O resultado numérico foi, à primeira vista, excelente: o erro de treino, que começou alto, despencou para praticamente zero depois de duzentas rodadas de treinamento. Só que o texto gerado ao final não fazia sentido nenhum: eram sílabas e fragmentos de palavras reais, mas sem coerência. Esse contraste é, na verdade, o achado mais interessante do experimento. Um erro de treino baixíssimo não significa que o modelo aprendeu a "escrever": significa apenas que ele decorou, quase letra por letra, o pequeno texto que viu repetidas vezes. É a diferença entre um estudante que decora as respostas de uma prova antiga e um estudante que realmente entendeu a matéria: o primeiro tira nota alta na mesma prova, mas se perde diante de uma pergunta nova. Modelos de grandes empresas reduzem significativamente esse problema ao serem treinados com volumes muito maiores e mais diversos de dados; aqui, o corpus pequeno ajudou a expor esse fenômeno de forma didática.

Construir esse modelo à mão, sem *framework* pronto, foi como desmontar um relógio mecânico só para entender as engrenagens: ninguém vai usar esse relógio artesanal no dia a dia, mas escrever cada cálculo de gradiente e cada matriz de atenção revela decisões de design que normalmente ficam escondidas atrás de uma única linha de código.

Para saber mais:

- https://github.com/MateusDBarros/neural-network
- https://arxiv.org/abs/1706.03762 (Attention Is All You Need)
- https://jalammar.github.io/illustrated-transformer/

Mateus de Barros é Desenvolvedor de Software, com 2 anos de experiência em tecnologia de informação, formando em 2027 pela Fatec Indaiatuba e membro do TLC-BR desde 2026. O **Mini Paper Series** é uma publicação quinzenal do **TLC-BR** e para assinar e receber eletronicamente as futuras edições, envie um e-mail para **tlcbr@br.ibm.com**.
Todas as informações e opiniões apresentadas neste artigo são de inteira responsabilidade do seu autor, e não refletem, necessariamente, a opinião ou posicionamento do seu empregador.